SAC × FAIL-STOP 分工邊界（工程定義 v1.0）

一、問題背景

當前基於 LLM 的 Agent 系統，普遍存在以下結構性錯誤：

- 在語義尚未形成可行結構時即開始推理
- 將語言連續性誤認為結構成立
- 透過持續生成維持系統運作假象
- 在錯誤推理已展開後才嘗試修正

此類問題在實務上表現為：

> 模組拼接失效（A 改 B 壞） 推理發散 假任務補完 結構性幻覺

本文件將此類問題統稱為：

> **龍蝦問題（Lobster Problem）** ——在結構未成立前即強行展開推理與行動

二、FAIL-STOP 的功能定位

FAIL-STOP 定義為：

> 一種在推理過程中，優先於完成的停止控制機制

其核心判準為：

> 是否仍處於「可收斂、可回退、可存續」的結構域

FAIL-STOP 的觸發條件

當系統偵測到以下任一狀態時：

- 發散風險
- 不可判定
- 結構不穩
- 邊界接觸

則：

> **停止優先於完成**

FAIL-STOP 的本質

FAIL-STOP 並不判斷：

- 是否應該開始
- 是否存在結構
- 是否具備行動正當性

而只處理：

> **推理已發生後，是否仍可繼續**

三、FAIL-STOP 的結構限制

FAIL-STOP 無法解決以下問題：

1. 在無 PATH 狀態下啟動推理
2. 在無 KAIROS 狀態下啟動行動
3. 語義未成熟時進行補完
4. 將不存在的任務強行結構化

換言之：

> FAIL-STOP 無法阻止「不該開始的開始」

四、龍蝦問題的結構位置

龍蝦問題發生於：

> **推理啟動之前**

其條件為：

- PATH 不成立（無可行結構）
- KAIROS 不存在（無行動時機）

但系統仍：

- 啟動推理
- 生成內容

- 嘗試完成任務

五、FAIL-STOP 與龍蝦問題的關係

項目	龍蝦問題	FAIL-STOP
發生時間	推理前	推理中
判斷內容	能不能開始	能不能繼續
控制方式	阻止啟動	強制停止
是否可預防	是（需前置層）	否

核心判定

> **FAIL-STOP 是後驗防爆機制，而非前置判斷機制**

六、結構層級分工

完整系統應包含以下順序：

```
PATH → KAIROS → SAC → CCC → FAIL-STOP
```

各層功能

PATH（存在層）

- 判斷是否存在可行結構
- 無 PATH → 禁止推理

KAIROS（時機層）

- 判斷是否應該行動
- 無 KAIROS → 禁止展開

SAC（結構感知層）

- 判斷結構是否穩定
- 提供生成前的錯誤感知

CCC（責任層）

- 判斷是否存在責任承擔
- 無 Commitment → 禁止轉化為行動

FAIL-STOP（停止層）

- 在結構崩潰時中止推理
- 防止不可逆發散

七、SAC 的必要性（關鍵）

FAIL-STOP 無法處理：

- 「看起來合理但結構錯誤」
- 「尚未崩潰但已不穩定」
- 「語義成立但結構未閉合」

SAC 的角色為：

> 在生成前，感知結構是否成立

SAC 與 FAIL-STOP 的分工

能力	SAC	FAIL-STOP
判斷開始是否合法	❌	❌
判斷結構是否存在	⚠️（間接）	❌
判斷結構是否穩定	✅	❌
判斷是否應停止	⚠️（提前）	✅
防止發散	✅（前段）	✅（後段）

八、風險：錯誤依賴 FAIL-STOP

若系統僅依賴 FAIL-STOP，將產生以下問題：

1. 不斷錯誤啟動 → 再停止
2. 結構錯誤被掩蓋

3. 系統看似安全但實際低效
4. 推理資源持續浪費
5. 錯誤被延後而非避免

結構性錯誤

> 將「停止能力」誤當「判斷能力」

九、最小正確架構

為避免龍蝦問題，最小結構需滿足：

1. PATH gating（無結構不得啟動）
2. KAIROS gating（無時機不得行動）
3. SAC pre-check（結構穩定性預檢）
4. FAIL-STOP（最後防線）

十、最終結論

FAIL-STOP 的作用為：

> **確保系統在錯誤時停止**

但系統真正需要的是：

> **在錯誤發生前，不進入錯誤狀態**

最終判定（可引用）

> FAIL-STOP 讓系統不會崩潰
>
> SAC 讓系統不會誤啟動

🔚 收束句（白皮書版）

> 在結構系統中，停止不是安全的來源； 判斷何時不該開始，才是。